BITS Pilani

Machine Learning

Anand Narasimhamurthy

innovate achieve lead

BITS Pilani

ALMLZ565, Machine Learning
Instance-based learning : An overview

Summary

Instance-based learning methods (often called lazy learners) simply store the training examples i.e. there is no explicit training.
When a new query instance is encountered, its relationship to the previously stored examples is examined in order to assign a target function value for the new instance.

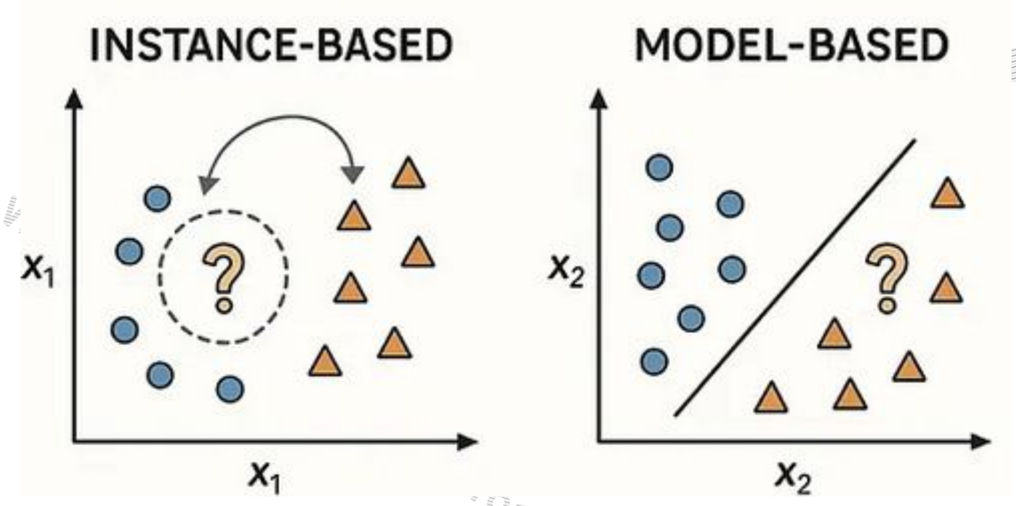

Image reproduced from https://medium.com/unlocking-ai/instance-based-vs-model-based-learning-19c5c64cf75d

Instance-based learning includes

- **Methods that assume instances can be represented as points in a Euclidean space.**

  Eg. nearest neighbor and locally weighted regression

Case Based Reasoning : A possibly more sophisticated and abstract way of using instances.

Distance-Weighted Nearest Neighbor Algorithm

Basic idea : Weight the contribution of each of the k neighbors according to their distance to the query point giving greater weight to closer neighbors.

Distance-Weighted Nearest Neighbor Algorithm

A refinement to knn is to weight the contribution of each of the k neighbors according to their distance to the query point giving greater weight to closer neighbors.

$$\hat{f}(x_q) \leftarrow \underset{v \in V}{\operatorname{argmax}} \sum_{i=1}^{k} w_i \delta(v, f(x_i))$$

Distance-Weighted Nearest Neighbor Algorithm

A common strategy is to make the weights inversely proportional to distance i.e.

$$w_i \equiv \frac{1}{d(x_q, x_i)^2}$$

where,

x_q is the query instance

Distance-Weighted Nearest Neighbor Algorithm

Distance-weight the instances for real-valued target functions can be done in a similar fashion

$$\hat{f}(x_q) \leftarrow \frac{\sum_{i=1}^{k} w_i f(x_i)}{\sum_{i=1}^{k} w_i}$$

Locally Weighted Regression

Locally weighted regression constructs an explicit approximation to target function f over a local region surrounding x_q using nearby or distance-weighted training examples to form this local approximation to f . Common possibilities are:

1. Minimize the squared error over just the k nearest neighbors:

$$E_1(x_q) \equiv \frac{1}{2} \sum_{x \in \, k \text{ nearest nbrs of } x_q} (f(x) - \hat{f}(x))^2$$

2. Minimize the squared error over the entire set D of training examples, while weighting the error of each training example by some decreasing function K of its distance from x_q:

$$E_2(x_q) \equiv \frac{1}{2} \sum_{x \in D} (f(x) - \hat{f}(x))^2 \; K(d(x_q, x))$$

3. Combine 1 and 2:

$$E_3(x_q) \equiv \frac{1}{2} \sum_{x \in \, k \text{ nearest nbrs of } x_q} (f(x) - \hat{f}(x))^2 \; K(d(x_q, x))$$

References and additional links

- **Nearest neighbor documentation on scikit learn**

https://scikit-learn.org/stable/modules/neighbors.html

- **Lecture notes on Nearest Neighbors,**

  Rich Zemel, Raquel Urtasun and Sanja Fidler

  University of Toronto,

https://www.cs.toronto.edu/~urtasun/courses/CSC411_Fall16/05_nn.pdf